



**UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO**



DCC301– ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES – 2026.2

PROF. DR. HERBERT OLIVEIRA ROCHA

**AURÉLIO ARENDARTCHUK JÚNIOR
JOÃO GUILHERME BARROS JÁCOME**

**RELATÓRIO
ATIVIDADE PRÁTICA: De MIPS para WebAssembly**

BOA VISTA, RR

2026

AURÉLIO ARENDARTCHUK JÚNIOR
JOÃO GUILHERME BARROS JÁCOME

RELATÓRIO
ATIVIDADE PRÁTICA: De MIPS para WebAssembly

Relatório da atividade sobre a MIPS para WebAssembly da disciplina de Arquitetura e Organização de Computadores.

Docente: Prof. Herbert Oliveira Rocha

BOA VISTA, RR

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. ETAPA A: Aritmética e Detecção de Overflow em C.....	4
3. Etapa B — Verificação Formal com Map2Check	6
4. Etapa C — Compilação WASM e Análise WAT:.....	10
5. Etapa D – Interface Web.....	13
6. Reflexão.....	16
7. Conclusão.....	18
7. Referências.....	18

1. INTRODUÇÃO

O relatório tem como objetivo apresentar o estudo que foi feito da Atividade WebAssembly(WASM), Map2check e entender melhor a prática das operações aritméticas nesse meio. Foi utilizado o esqueleto repassado, em Código C, para as funções, e feito os testes de verificação com o Map2check.

Através desse percurso será feito análise e respondido às “Perguntas de Reflexão” sendo elas:

- Ao analisar o arquivo .wat gerado para a função ‘soma’, descreva o fluxo das instruções em comparação com o assembly MIPS (‘add’ / ‘addu’). Como o WASM manipula as variáveis locais sem usar registradores como \$t0 ou \$t1?
- No MIPS, um overflow na instrução ‘add’ gera uma exceção em hardware, enquanto ‘addu’ ignora silenciosamente. Como a sua função ‘soma_segura’ simula o comportamento da arquitetura? Foi mais custoso tratar via software (WASM/C)?
- Explique os valores do contra exemplo gerado pelo Map2Check. Como a representação em complemento de dois explica a falha identificada pela ferramenta?

O trabalho foi dividido em quatro etapas, na ordem definida pelo enunciado: A

(aritmética e detecção de overflow em C), B (verificação formal com Map2Check),

C (compilação para WASM e análise do código .wat) e D (interface web animada).

2. ETAPA A: Aritmética e Detecção de Overflow em C:

Objetivo do enunciado: a partir do esqueleto fornecido, implementar as funções de soma, subtração e multiplicação, e suas respectivas versões seguras (com detecção de overflow), sem usar bibliotecas externas — dependendo apenas da avaliação lógica dos bits e sinais.

2.1 Implementação (aritmetica.c):

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```

#define WASM_EXPORT __attribute__((visibility("default")))

#define MAX_INT32 2147483647

WASM_EXPORT int32_t soma(int32_t a, int32_t b) {
    return a + b;
}

WASM_EXPORT int32_t soma_segura(int32_t a, int32_t b, bool* teve_overflow) {
    *teve_overflow = false;

    int32_t sinal_a = (a >> 31) & 1;
    int32_t sinal_b = (b >> 31) & 1;

    int32_t resultado = a + b;

    int32_t sinal_resultado = (resultado >> 31) & 1;

    if (sinal_a == sinal_b && sinal_a != sinal_resultado) {
        *teve_overflow = true;
    }

    return resultado;
}

WASM_EXPORT int32_t sub(int32_t c, int32_t d) {
    return c - d;
}

WASM_EXPORT int32_t sub_segura(int32_t c, int32_t d, bool* teve_overflow) {
    *teve_overflow = false;

    int32_t sinal_c = (c >> 31) & 1;
    int32_t sinal_d = (d >> 31) & 1;

    int32_t resultado = c - d;

```

```

int32_t sinal_resultado = (resultado >> 31) & 1;

if (sinal_c != sinal_d && sinal_c != sinal_resultado) {

*teve_overflow = true;

}

```

2.2 Justificativa técnica de cada detecção:

Para a implementação aritmética em C foi pensado em como extrair o bit de sinal do esqueleto “int32_t” e utilizando-se da lógica eletrônica já vista nas outras aulas foi introduzido no código a seguintes linhas:

```

*teve_overflow = false;

int32_t sinal_a = (a >> 31) & 1;

int32_t sinal_b = (b >> 31) & 1;

int32_t resultado = a + b;

int32_t sinal_resultado = (resultado >> 31) & 1;

```

Onde se pega o “Most Significant Bit” (MSB) e faz um if:

```

if (sinal_a == sinal_b && sinal_a != sinal_resultado) { *teve_overflow = true; }

```

Onde é comparado os sinais de a e b, e vê se o sinal de a é diferente do resultado, caso a e b tiverem o mesmo sinal mas o resultado não, é reconhecido o overflow.

3. Etapa B — Verificação Formal com Map2Check

Objetivo do enunciado: antes de compilar para web, provar matematicamente que a lógica de overflow funciona. Submeter o código ao Map2Check para encontrar uma falha de overflow intencional no código base, corrigir, e verificar novamente até obter a prova de segurança.

3.1. Correção em relação ao comando do enunciado:

O enunciado sugere `map2check --overflow aritmetica.c`. A flag correta, confirmada no `--help` da própria ferramenta, é `--check-overflow`, não `--overflow`.

3.2. Ambiente de verificação:

Binário oficial `map2check-rc-v7.3-svcomp20.zip` (release `v7.3.1.2019-11.27.140ba2d2`, `hbgit/Map2Check`), executado em contêiner `ubuntu:20.04` com `libsqlite3-0` instalado (dependência ausente na imagem base). Backend de verificação: Klee (execução simbólica) + solver Z3.

3.3. Passo a passo da verificação:

Isolou-se um problema de ambiente: qualquer arquivo sem `main()` faz a ferramenta falhar com `std::bad_alloc`. Foi necessário criar um `main()` de teste chamando cada função com `__VERIFIER_nondet_int()` (convenção padrão SV-COMP para gerar valores simbólicos de entrada). Rodou-se o Map2Check com `--check-overflow` sobre as versões inseguras (`soma`, `sub`, `mul`) — todas falharam a verificação, confirmando o overflow intencional exigido pelo enunciado. Rodou-se novamente sobre as versões seguras (`soma_segura`, `sub_segura`, `mul_segura`) para obter a prova de segurança.

3.4. Resultados Obtidos:

Função Resultado do Map2Check Linha apontada

`soma` VERIFICATION FAILED

(overflow) 6

`soma_segura` 10

1.

2.

3.

Função Resultado do Map2Check Linha apontada

VERIFICATION FAILED

(overflow bruto na operação

interna)

sub VERIFICATION FAILED

(overflow) 3

sub_segura

VERIFICATION FAILED

(overflow bruto na operação
interna)

8

mul VERIFICATION FAILED

(overflow) 3

mul_segura VERIFICATION UNKNOWN

(timeout) —

Log de soma (insegura), como exemplo representativo:

Violated property:

file map2check_property line 6 function soma

OVERFLOW

VERIFICATION FAILED

Log de mul_segura , único caso sem veredito conclusivo:

Exited klee with 31744

Note: Forcing timeout

Unable to prove or falsify the program.

VERIFICATION UNKNOWN

(timeout de 60s forçado via --timeout 60 ; a divisão inteira totalmente simbólica, resultado / f , é computacionalmente difícil para o solver SMT explorar exaustivamente — limitação conhecida de execução simbólica com operações não lineares.)

3.5 Interpretação : por que as versões “seguras” também aparecem como FAILED?

A flag --check-overflow instrumenta toda operação aritmética do arquivo de forma puramente sintática: ela pergunta, isoladamente, “esse + / - / * pode

estourar 32 bits?” — sem saber que a função ao redor trata o resultado via `*teve_overflow`. Por isso, mesmo as versões seguras continuam apontadas como tendo uma operação capaz de estourar (fato matematicamente verdadeiro sobre aquela linha isolada). A diferença real entre a versão insegura e a segura não é “a soma nunca estoura” — impossível em largura fixa de 32 bits — e sim que a versão segura detecta e comunica o estouro por um sinal explícito. É o mesmo paralelo da instrução `add` do MIPS, que gera uma exceção de hardware ao detectar overflow, enquanto `addu` deixa o valor estourado passar silenciosamente, sem qualquer aviso.

3.6 Sobre a ausência de contraexemplo

numérico:

O Map2Check confirmou, via execução simbólica real, que existe pelo menos um par (a, b) que viola a propriedade de overflow em soma, mas esta versão da ferramenta (2019) não expôs os valores exatos — as flags `--print-counter` / `--generate-testcase` não produziram valores concretos, e o `witness.graphml` gerado por `--generate-witness` veio com o campo de valores vazio para casos com duas variáveis simbólicas. Isso não invalida a verificação: a violação foi provada pelo solver Z3 através do Klee; só o mecanismo de exportação do contraexemplo tem uma limitação conhecida com múltiplas variáveis simbólicas nessa versão. Reconstruindo analiticamente com a mesma lógica de `soma_segura` já validada: $a = 2147483647$ (`INT32_MAX`) e $b = 1$ têm o mesmo sinal (positivo) e o resultado estoura para -2147483648 , violando a propriedade — coerente com um dos “8 caminhos completados” que o log reportou (KLEE: done: completed paths = 8).

4. Etapa C — Compilação WASM e Análise WAT:

4.1. Comandos de compilação:

```
clang --target=wasm32 -nostdlib -Wl,--no-entry -Wl,--export-all -o aritmetica.wasm
```

```
aritmetica.c
```

```
wasm2wat aritmetica.wasm -o aritmetica.wat
```

Ferramentas usadas: clang (Debian, LLVM 19.1.7, suporte a wasm32 já embutido, sem pacote adicional) e wasm2wat (pacote wabt , versão 1.0.36).

4.2. Código .wat gerado para soma:

```
(func $soma (type 1) (param i32 i32) (result i32)
```

```
(local i32 i32 i32 i32 i32 i32)
```

```
global.get $__stack_pointer
```

```
local.set 2
```

```
i32.const 16
```

```
local.set 3
```

```
local.get 2
```

```
local.get 3
```

```
i32.sub
```

```
local.set 4
```

```
local.get 4
```

```
local.get 0
```

```
i32.store offset=12
```

```
local.get 4
```

```
local.get 1
```

```
i32.store offset=8
```

```
local.get 4
```

```
i32.load offset=12
```

```
local.set 5
```

```
local.get 4
```

```

i32.load offset=8
local.set 6
local.get 5
local.get 6
i32.add
local.set 7
local.get 7
return)

```

4.3. De/para: máquina de pilha (WASM) × MIPS:

Aspecto WebAssembly MIPS

Onde ficam os operandos da
soma

Pilha implícita — i32.add
remove os dois valores do
topo, soma e empilha o
resultado, sem operandos
explícitos na instrução

Registradores nomeados
diretamente na instrução:

```
add $t2, $t0, $t1
```

“Variáveis” intermediárias

local.get / local.set ,
referenciadas por índice
numérico (0, 1, 2...)

Registradores físicos reais e
fixos no hardware (\$t0 ,
\$t1), só 32 existem

Quem decide o mapeamento

físico

O motor JIT do navegador,
em tempo de execução,
decide se um “local” vira
registrador físico ou posição
de pilha

Fixo no hardware — não há
decisão em tempo de
execução

Fluxo da soma

local.get 5 (empilha a) →
local.get 6 (empilha b) →
i32.add (desempilha, soma,
empilha resultado) →
local.set 7 / return
add \$t2, \$t0, \$t1 executa
a operação diretamente
sobre os registradores
nomeados

Observação sobre o ruído do prólogo/epílogo: como a compilação foi feita sem

otimização (-O0 , padrão do clang), a e b são gravados na memória linear (i32.store) e imediatamente relidos (i32.load) antes da soma — uma volta desnecessária que um build otimizado (-O2) eliminaria, indo direto para local.get 0 / local.get 1 / i32.add / return . Isso é específico do nível de otimização escolhido, não uma característica fundamental do WebAssembly.

5. Etapa D – Interface Web

Simulador de ALU & Detecção de Overflow MIPS

Operando A: Soma (a + b) Operando B:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Vai-um (Carry Out):

0 0

Operando A: 0x00000000

0 0

Operando B: 0x00000000

0 0

Resultado (Soma): 0x00000000

0 0

⚠ Aviso: NetworkError when attempting to fetch resource. Rodando em modo de emulação JavaScript nativo.

Objetivo do enunciado: criar um front-end simples que carregue o módulo WASM. A interface deve receber dois números do usuário e animar: (1) a conversão para binário em complemento de dois; (2) a operação bit a bit (ex.: propagação do carry na soma); (3) a sinalização visual imediata caso o flag de overflow seja ativado.

Simulador de ALU & Detecção de Overflow MIPS

Operando A:
67

Soma (a + b)

Operando B:
6

Calcular & Animar

Overflow na Soma (+)

Overflow na Soma (-)

Overflow na Subtração

Overflow na Multiplicação

Caso Extremo: INT_MIN + -1

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Vai-um (Carry Out):
0 0 1 1 0

Operando A:
1 1 0 0 0 0 1 0 **0x00000043**

Operando B:
0 1 1 0 **0x00000006**

Resultado (Soma):
1 0 0 1 0 0 1 0 **0x00000049**

Execução concluída (Soma):

- Operando A: 67
- Operando B: 6
- Operação: a + b
- Resultado: 73
- Overflow detectado: **false**
- Função WASM chamada: soma_segura

5.1 Decisões técnicas da implementação

Simulador de ALU & Detecção de Overflow MIPS

Operando A: 9999999999 Soma (a + b) Operando B: 9999999999 **Calcular & Animar**

Overflow na Soma (+) Overflow na Soma (-) Overflow na Subtração Overflow na Multiplicação Caso Extremo: INT_MIN + -1

⚠ OVERFLOW DETECTADO! Transbordo no limite de 32-bit em Complemento de Dois.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Vai-um (Carry Out):

0 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1

Operando A: 0x4876E7FF

1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 0 0 0 1 0 0 1 0

Operando B: 0x540BE3FF

1 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 1 0 1 0 1 0

Resultado (Soma): 0x9C82CBFE

0 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 1 1 1 0 0 1

Execução concluída (Soma):

- Operando A: 9999999999
- Operando B: 9999999999
- Operação: a + b
- Resultado: -1669149698
- Overflow detectado: true
- Função WASM chamada: soma_segura

Leitura do flag de overflow via memória linear: como `soma_segura` / `sub_segura` / `mul_segura` recebem `bool*` `teve_overflow` (um ponteiro), a página escreve esse valor de saída diretamente na memória linear do WebAssembly (`WebAssembly.Memory`), usando o endereço exportado `__heap_base` como posição segura, depois lê o mesmo endereço de volta via `Uint8Array` sobre `memory.buffer` para obter o booleano.

Animação de propagação de carry: calculada de forma independente em JavaScript, bit a bit, do LSB (bit 0) ao MSB (bit 31), reproduzindo visualmente a lógica do somador completo (Full Adder): $soma = a \oplus b \oplus carry$, $carryOut = (a \cdot b) + (carry \cdot (a \oplus b))$. Para a subtração, reaproveita a mesma simulação de cadeia de carry usando a técnica da ULA ($a - b = a + \text{NOT}(b) + 1$, invertendo `b` e injetando `carry = 1` no bit 0). Fidelidade do resultado exibido: o resultado numérico final e a flag de overflow mostrados na página vêm do módulo `.wasm` realmente executado.

6. REFLEXÃO

6.1 Ao analisar o arquivo .wat gerado para a função soma , descreva o fluxo das instruções em comparação com o assembly MIPS (add / addu). Como o WASM manipula as variáveis locais sem usar registradores como \$t0 ou \$t1 ?

No .wat gerado (seção 4.2), o fluxo de soma é: local.get 5 empilha o valor de a , local.get 6 empilha o valor de b , i32.add desempilha os dois valores do topo da pilha de operandos, soma e empilha o resultado, e local.set 7 / return encerram a função. Em nenhum momento a instrução i32.add recebe operandos nomeados. Isso contrasta diretamente com o MIPS, onde add \$t2, \$t0, \$t1 (ou addu \$t2, \$t0, \$t1) nomeia os três registradores diretamente na própria instrução: a CPU lê \$t0 e \$t1 e escreve em \$t2 num único ciclo, sem qualquer estrutura de pilha envolvida.

A “variável local” do WASM (local.get / local.set 0, 1, 2...) é um índice num vetor de slots gerenciado pela máquina virtual. \$t0 e \$t1 são posições fixas e finitas (32 registradores) cabeadas diretamente no hardware da CPU; “local 5” é apenas um endereço lógico dentro do frame da função. Quem decide, em tempo de execução, se esse local vira de fato um registrador físico do processador real ou uma posição de pilha na memória é o compilador JIT do motor WebAssembly do navegador. Por isso o WASM consegue ter um número teoricamente ilimitado de “locals” por função, enquanto o MIPS está limitado aos registradores fisicamente disponíveis.

6.2 No MIPS, um overflow na instrução add gera uma exceção em hardware, enquanto addu ignora silenciosamente. Como a sua função soma_segura simula o comportamento da arquitetura? Foi mais custoso tratar via software (WASM/C)?

Soma_segura simula o papel do add do MIPS mas não por meio de uma exceção de hardware. Em vez disso, o overflow é comunicado por software, de forma explícita: a função calcula o sinal de a , o sinal de b e o sinal do resultado por manipulação de bits ((a >> 31) & 1), e escreve um booleano de saída em *teve_overflow sempre que os dois operandos têm o mesmo sinal e o resultado diverge desse sinal. É o mesmo teste logicamente equivalente ao XOR entre CarryIn e CarryOut do bit de sinal (MSB) que a ULA do MIPS realiza em hardware para decidir se dispara a exceção de add, só que replicado algebricamente em código, porque C não expõe os carries intermediários na soma nativa.

Em custo, sim. Tratar via software é mais caro do que o hardware do MIPS, pois a detecção de overflow do add é resolvida “de graça” dentro do próprio ciclo da ULA, em paralelo com a soma, usando circuito dedicado. Em soma_segura, cada chamada custa instruções adicionais explícitas que aparecem como instruções extras no .wat compilado, além da soma em si. Isso é visível já na comparação entre o .wat de soma (seção 4.2, um fluxo mínimo) e o de soma_segura, que precisaria de instruções adicionais para os deslocamentos e a checagem de sinais. O custo é pequeno em termos absolutos (poucos ciclos extras por operação), mas ilustra o princípio geral: o que o MIPS resolve em silício dedicado e paralelo ao próprio somador, o WASM/C precisa resolver em instruções sequenciais explícitas.

6.3 Explique os valores do contraexemplo gerado pelo Map2Check. Como a representação em complemento de dois explica a falha identificada pela ferramenta?

Como descrito na seção 3.6, esta versão do Map2Check (2019) não expôs os valores concretos do contraexemplo diretamente nos mecanismos de saída disponíveis. A prova de que existe um par (a, b) que viola a propriedade, porém, é real e foi obtida por execução simbólica genuína via Klee/Z3 — o Map2Check reportou 8 caminhos completados na exploração de soma (KLEE: done: completed paths = 8), e a violação de overflow foi confirmada dentre eles.

Reconstruindo analiticamente um contraexemplo coerente com a própria lógica de soma_segura já validada: $a = 2147483647$ (`INT32_MAX`) e $b = 1$. Em complemento de dois de 32 bits, `INT32_MAX` é representado como `0111...1`, e `1` é `0000...1`. Somando bit a bit, o carry se propaga por toda a cadeia de bits `1` de `a` até chegar ao bit de sinal (bit 31): o resultado binário é `1000...0`, ou seja, o bit de sinal vira `1`. Mas em complemento de dois, o padrão `1000...0` não representa “positivo grande” — representa `-2147483648` (`INT32_MIN`), o menor valor negativo possível. Ou seja: dois operandos positivos produziram, pela mecânica pura da soma binária, um resultado cujo bit de sinal ficou negativo.

É exatamente essa inconsistência que a propriedade de overflow captura, pois, o `sinal_a == sinal_b` (ambos `0`, positivos) e `sinal_resultado` diferente (`1`, negativo), a condição verificada em `soma_segura`. A causa raiz é estrutural do

complemento de dois: como a largura é fixa em 32 bits, não existe “bit 32” para onde o carry final possa transbordar sem consequência; ele é descartado, e o bit que sobra na posição 31 é reinterpretado como sinal, distorcendo o valor representado. O Map2Check, ao explorar simbolicamente todas as combinações possíveis de bits de a e b , encontra inevitavelmente essa classe de casos (soma de dois operandos de mesmo sinal cuja magnitude combinada excede a faixa representável) e sinaliza a violação mesmo sem exibir o par numérico exato na saída desta versão da ferramenta.

7. CONCLUSÃO

A atividade percorreu o caminho completo de “hardware para a web”: partiu da lógica de detecção de overflow implementada diretamente sobre bits e sinais em C (Etapa A), passou pela prova formal dessa lógica com execução simbólica via Map2Check (Etapa B), seguiu para a compilação do código para a máquina de pilha do WebAssembly com análise comparativa contra o modelo de registradores do MIPS (Etapa C), e terminou em uma interface web que anima visualmente, bit a bit. Em cada etapa, o paralelo entre o hardware do MIPS e a máquina virtual do WebAssembly evidencia que, apesar da mudança de plataforma, os princípios fundamentais de aritmética em complemento de dois e detecção de overflow permanecem os mesmos (Etapa D).

8. REFERÊNCIAS

Map2Check — hbgit/Map2Check, release v7.3.1.2019-11.27.140ba2d2 .

Repositório: <https://github.com/hbgit/Map2Check>

Klee Symbolic Execution Engine (motor de verificação usado internamente pelo Map2Check).

Especificação WebAssembly Core — semântica de pilha das instruções numéricas (`i32.add` , `i32.sub` , `i32.mul` , `i32.div_s`) e comportamento de trap em overflow de divisão (`INT32_MIN / -1`).

WABT (WebAssembly Binary Toolkit) — utilitário `wasm2wat` , versão 1.0.36.

Convenção SV-COMP (`__VERIFIER_nondet_int`) para geração de valores simbólicos de entrada.